

Versão completa — fluxo Apresentar Agenda → Gravar Medware (parecer 5 engenheiros)

Origem: Fábio 05/06/2026 — convocou 5 engenheiros especializados pra dar parecer técnico e entregar versão funcional pronta de cabo a rabo. Sintetiza diagnóstico + código real.

Síntese dos pareceres

#	Engenheiro	Veredito central
1	Arquiteto de Agentes IA	Tool calling existe em <code>responder.py:1971</code> mas só dispara via <code>_agenda_ctx</code> presente. Falha tripla: (a) <code>tool_choice={"type":"any"}</code> permite escolher tool errada, (b) não força por estado FSM, (c) Sonnet dilui atenção em prompt de 15k tokens. Solução: filtrar tools por estado + <code>tool_choice</code> específico + agente dedicado de agendamento (router→specialist) com Haiku 4.5.
2	Filtros Defensivos	Filtros regex são backstop bom mas têm cauda longa infinita. 3 novos urgentes: (a) <code>_viola_dia_semana_estendido</code> (cobre data sem dia-semana), (b) <code>_viola_cronologia_total</code> (compara TODAS datas vs agenda real), (c) <code>_viola_unidade_turno</code> (Águas Claras nunca noite). E <code>_viola_frase_exemplo</code> pra Adelia. Quando >3 bugs do mesmo TIPO, migrar para tool calling — passou o limite.
3	Medware/ Integração	Fluxo end-to-end com <code>criar_agendamento_seguro</code> unificado: dedup Redis 24h, validações duras (médico em pausa, convênio bloqueado, CPF dígito verificador), slot ∈ agenda real (bug Noah), tratamento CPF duplicado + 409 conflito, sync Kommo APÓS sucesso. Pediátrico: CPF é DO paciente (bebê tem CPF desde 2010).
4	DevOps/ Observabilidade	Bug #240 = User-Agent ausente (probabilidade 60%) — <code>_headers</code> em <code>kommo.py</code> não envia UA, WAF Kommo bloqueia. Fix <code>patch_custom_fields_raw</code> com validação GET de verdade. Endpoint <code>/admin/Leads-abandonados</code> + cron 5min + Slack pra capturar lead 24107106. CI smoke prod antes de marcar verde.
5	QA/Pytest	E2E parametrizado 8 cenários (<code>test_e2e_apresentar_e_gravar.py</code>), property-based Hypothesis blindando data×weekday (100+ combos), smoke contínuo PROD via <code>/admin/simulate-inbound</code> 5 cenários core, pre-commit hook que rejeita <code>fix(bug):</code> sem teste com <code>lead_id</code> real.

Versão completa do código (pronta pra implementar)

Arquivo 1 — `voice_agent/responder.py` (PATCH)

Substituir linhas 1971-1987 (loop `tool_use`) por:

```

from voice_agent.tools_lia import (
    TOOL_OFERECER_SLOT, TOOL_CONFIRMAR_DADOS_PACIENTE,
    TOOL_GRAVAR_AGENDAMENTO_MEDWARE,
)

_ctx = caller_context or {}
_fsm_estado = (_ctx.get("fsm") or {}).get("estado", "TRIAGEM")
_agenda_ctx = _ctx.get("agenda") or []
_ja_agendado = _ctx.get("ja_agendado")

# Tools filtradas por estado FSM – reduz superfície do roteador interno do Sonnet
if _fsm_estado == "AGENDA" and not _ja_agendado:
    _tools_iter = [TOOL_OFERECER_SLOT]
    _force_tool_kwargs = (
        {"tool_choice": {"type": "tool", "name": "oferecer_slot"}}
        if _agenda_ctx else {"tool_choice": {"type": "any"}}
    )
elif _fsm_estado == "CONFIRMACAO" and _agenda_ctx:
    _tools_iter = [TOOL_GRAVAR_AGENDAMENTO_MEDWARE, TOOL_OFERECER_SLOT]
    _force_tool_kwargs = {"tool_choice": {"type": "tool", "name": "gravar_agendamento_medware"}}
elif _fsm_estado == "DADOS":
    _tools_iter = [TOOL_CONFIRMAR_DADOS_PACIENTE]
    _force_tool_kwargs = {"tool_choice": {"type": "any"}}
else:
    _tools_iter = ALL_TOOLS
    _force_tool_kwargs = {}

for _iter in range(4):
    _iter_kwargs = _force_tool_kwargs if _iter == 0 else {}
    response = self._client.messages.create(
        model=model, max_tokens=600, system=system_field,
        messages=messages_acc, temperature=0.3,
        tools=_tools_iter, **_iter_kwargs,
    )
    # Loop processar response.stop_reason == "tool_use" → executar handler →
    # resposta humanizada SÓ COM dados retornados pela tool
    ...

```

Arquivo 2 — `voice_agent/pipeline.py` (lock Redis)

No topo de `process_inbound()`:

```

LOCK_TTL = 30
lock_key = f"blink:lock_pipeline:{conversation_key}"
acquired = redis.set(lock_key, str(time.time()), nx=True, ex=LOCK_TTL)
if not acquired:
    log.warning("[LOCK] convo=%s ocupada – buferizando", conversation_key)
    redis.rpush(f"blink:pending:{conversation_key}", user_text)
    redis.expire(f"blink:pending:{conversation_key}", 120)
    return PipelineResult(sent=False, deferred=True)
try:
    pending = redis.lrange(f"blink:pending:{conversation_key}", 0, -1)
    if pending:
        redis.delete(f"blink:pending:{conversation_key}")
        user_text = "\n".join([p.decode() if isinstance(p, bytes) else p for p in pending]) + "\n" + user_text
        # ... processamento normal
finally:
    redis.delete(lock_key)

```



```

import re
from datetime import datetime, timezone, timedelta
from typing import Tuple

_TZ_BR = timezone(timedelta(hours=-3))
_DIA_PT = ["segunda-feira", "terça-feira", "quarta-feira", "quinta-feira", "sexta-feira", "sábado", "domingo"]
_RE_DATA = re.compile(r"(?!\\d)(\\d{1,2})/(\\d{1,2})(?:/(\\d{2,4}))?(?!\\d)")
_RE_HORA = re.compile(r"\\b(\\d{1,2})[h:] (\\d{0,2})\\b")

_UNIDADE_HORARIOS = {
    "asa norte": {"min": 7, "max": 19},
    "águas claras": {"min": 7, "max": 18},
    "aguas claras": {"min": 7, "max": 18},
}

_FRASES_EXEMPLO_BANIDAS = [
    r"(?i)\\[(nome|paciente|m[éé]dico|unidade|data|hora)\\]",
    r"(?i)\\{\\{[^}]+\\}\\}",
    r"(?i)exemplo\\s*[: :]\\s*['\""]",
    r"(?i)por\\s+exemplo\\s*[: :]\\s*['\""]",
]

_RE_FRASE_EXEMPLO = re.compile("|".join(_FRASES_EXEMPLO_BANIDAS))

def _viola_frase_exemplo(text, ctx=None) -> Tuple[bool, str]:
    """Bug Adelia 24056883: Lia copiou placeholder/exemplo do prompt."""
    if not text: return (False, "")
    if _RE_FRASE_EXEMPLO.search(text):
        return (True, "copiou_placeholder_do_prompt")
    return (False, "")

def _viola_dia_semana_estendido(text, ctx) -> Tuple[bool, str]:
    """Estende Bug Priscila: valida data SEM dia-semana também."""
    if not text or not ctx: return (False, "")
    medico = (ctx.get("medico") or "").lower()
    permitidos = {"karla": {0,1,2,3,4}, "fabricio": {1,3}, "fabrício": {1,3}}
    medico_key = next((k for k in permitidos if k in medico), None)
    if not medico_key: return (False, "")
    hoje = datetime.now(_TZ_BR).date()
    for m in _RE_DATA.finditer(text):
        try:
            d, mm = int(m.group(1)), int(m.group(2))
            yy = int(m.group(3)) if m.group(3) else hoje.year
            if yy < 100: yy += 2000
            dt = datetime(yy, mm, d).date()
            if (hoje - dt).days > 30: dt = datetime(yy+1, mm, d).date()
        except (ValueError, TypeError):
            return (True, "data_invalida")
    if dt.weekday() not in permitidos[medico_key]:
        return (True, f"medico_{medico_key}_nao_atende_{_DIA_PT[dt.weekday()]}_{dt.strftime('%d/%m')}")
    return (False, "")

def _viola_cronologia_total(text, ctx, tolerancia_dias: int = 5) -> Tuple[bool, str]:
    """Compara TODAS as datas ofertadas vs TODA a agenda real."""
    from voice_agent.filtros_pedro_miguel import extrair_datas_oferecidas, menor_data_na_agenda
    if not text or not ctx or not ctx.get("agenda"): return (False, "")
    mais_proxima = menor_data_na_agenda(ctx["agenda"])
    if not mais_proxima: return (False, "")
    ofertadas = extrair_datas_oferecidas(text)
    if not ofertadas: return (False, "")
    pref = ctx.get("preferencia_data")
    ref = pref or mais_proxima
    melhor = min(ofertadas)
    if (melhor - ref).days > tolerancia_dias and (mais_proxima - ref).days <= tolerancia_dias:
        return (True, f"ofertou_{melhor.strftime('%d/%m')}_tinha_{mais_proxima.strftime('%d/%m')}")
    return (False, "")

```

```

def _viola_unidade_turno(text, ctx) -> Tuple[bool, str]:
    """Águas Claras NUNCA noite. Asa Norte máx 19h."""
    if not text or not ctx: return (False, "")
    unidade = (ctx.get("unidade") or "").lower()
    regra = next((v for k, v in _UNIDADE_HORARIOS.items() if k in unidade), None)
    if not regra: return (False, "")
    for m in _RE_HORA.finditer(text):
        try:
            h = int(m.group(1))
            if h < regra["min"] or h >= regra["max"]:
                return (True, f"unidade_{unidade}_nao_atende_{h}h")
        except ValueError:
            continue
    return (False, "")

def validar_oferta_de_slot(text, ctx) -> Tuple[bool, str]:
    """Pipeline unificado. Retorna (valido, motivo_falha). Curto-circuita.
    Ordem: frase_exemplo (mais barato) → dia_semana → unidade_turno → cronologia.
    """
    import logging
    log = logging.getLogger(__name__)
    pipeline = [
        ("frase_exemplo", _viola_frase_exemplo),
        ("dia_semana", _viola_dia_semana_estendido),
        ("unidade_turno", _viola_unidade_turno),
        ("cronologia", _viola_cronologia_total),
    ]
    for nome, fn in pipeline:
        try:
            violou, motivo = fn(text, ctx)
        except Exception as e:
            log.exception(f"filtro {nome} crashou: {e}")
            continue
        if violou:
            return (False, f"{nome}:{motivo}")
    return (True, "")

```



```

import json, time, logging
from voice_agent.checklist_dados_minimos import cpf_ok, data_nascimento_ok, nome_completo_ok
log = logging.getLogger(__name__)

CONVENIOS_BLOQUEADOS = {"inas", "inas gdf", "gdf", "sulamerica", "sul america", "bradesco", "cass"}
MEDICOS_EM_PAUSA = {"katia", "kátia"}

def criar_agendamento_seguro(args, ctx, medware_client, kommo_client, redis_client):
    """Fluxo unificado: validações + dedup + Medware + sync Kommo."""
    known = ctx.get("known") or {}
    convo_key = ctx.get("conversation_key", "")
    lead_id = ctx.get("lead_id")

    # 1. Dedup Redis 24h
    if redis_client and convo_key:
        ja = redis_client.get(f"blink:agendamento_gravado:{convo_key}")
        if ja:
            payload = json.loads(ja.decode() if isinstance(ja, bytes) else ja)
            return ResultadoTool(
                texto_para_paciente=args.get("mensagem_humana", ""),
                efeitos_colaterais=[f"dedup: cod_ag={payload.get('cod_agendamento')}]",
                tool_name="criar_agendamento_seguro",
            )

    # 2. Validações duras
    medico = (known.get("medico") or "").strip().lower()
    if any(m in medico for m in MEDICOS_EM_PAUSA):
        return ResultadoTool(texto_para_paciente="", erro="medico_em_pausa: escalar humano")
    conv = (known.get("convenio") or "").strip().lower()
    if any(b in conv for b in CONVENIOS_BLOQUEADOS):
        return ResultadoTool(texto_para_paciente="", erro="convenio_bloqueado: ofertar particular")
    if not nome_completo_ok(known.get("nome_paciente", "")):
        return ResultadoTool(texto_para_paciente="", erro="nome_invalido")
    if not data_nascimento_ok(known.get("data_nasc", "")):
        return ResultadoTool(texto_para_paciente="", erro="data_nasc_invalida")
    if not cpf_ok(known.get("cpf", "")):
        return ResultadoTool(texto_para_paciente="", erro="cpf_invalido")

    # 3. Slot € agenda real (bug Noah 04/06)
    slot = (args["data_iso"], args["hora"])
    agenda_set = {(s["data_iso"], s["hora"]) for s in (ctx.get("agenda") or [])}
    if agenda_set and slot not in agenda_set:
        return ResultadoTool(texto_para_paciente="", erro="slot_nao_existe_no_medware")

    # 4. Medware (retry/breaker já em criar_agendamento)
    res = medware_client.criar_agendamento(
        cod_medico=cod_medico_por_nome(known.get("medico", "")),
        cod_unidade=cod_unidade_por_nome(known.get("unidade", "")),
        cod_agenda=int(args.get("cod_agenda") or 0),
        data_hora=f"{args['data_iso']}T{args['hora']}",
        nome=known["nome_paciente"], cpf=known["cpf"],
        data_nascimento=known["data_nasc"], celular=known.get("telefone", ""),
        convenio=known.get("convenio"),
        obs=f"Lia · conv {convo_key}",
    )
    if not res.get("ok"):
        log.error("[GRAV] FAIL convo=%s motivo=%s", convo_key, res.get("motivo"))
        return ResultadoTool(texto_para_paciente="", erro=f"medware: {res.get('motivo')}")

    cod_ag = res["cod_agendamento"]
    log.info("[GRAV] OK convo=%s cod_ag=%s slot=%s", convo_key, cod_ag, slot)

    # 5. Dedup + Sync Kommo APÓS sucesso
    if redis_client and convo_key:
        redis_client.setex(
            f"blink:agendamento_gravado:{convo_key}", 86400,
            json.dumps({"cod_agendamento": cod_ag, **args}),
        )

```

```

_sync_kommo_pos_gravacao(kommo_client, lead_id, cod_ag, args, known)
return ResultadoTool(
    texto_para_paciente=args.get("mensagem_humana", ""),
    efeitos_colaterais=[f"MEDWARE cod_ag={cod_ag}", "kommo_sync_ok"],
    tool_name="criar_agendamento_seguro",
)

def _sync_kommo_pos_gravacao(kommo, lead_id, cod_ag, args, known):
    """Move pra 5-AGENDADO + 5 campos + nota."""
    ts = int(time.time())
    data_consulta_ts = int(time.mktime(time.strptime(
        f"{args['data_iso']} {args['hora']}", "%Y-%m-%d %H:%M"
    )))
    cfs = [
        {"field_id": 1255723, "values": [{"value": data_consulta_ts}], # 1.DIA CONSULTA
        {"field_id": 1260854, "values": [{"value": "agendado_aguarda_consulta"}]},
        {"field_id": 1260858, "values": [{"value": "confirmar_horario_d-1"}]},
        {"field_id": 1260856, "values": [{"value": f"[LIA] Agendado {args['data_iso']} {args['hora']} cod_ag={cod_ag}"}]},
        {"field_id": 1260860, "values": [{"value": ts}]}
    ]
    kommo.patch_custom_fields_raw(lead_id, cfs)
    kommo.move_lead_to_status(lead_id, 101507507) # 5-AGENDADO
    kommo.add_note(
        lead_id,
        f"AGENDAMENTO MEDWARE OK\ncod_ag={cod_ag}\nmédico={known['medico']}\n"
        f"unidade={known['unidade']}\nslot={args['data_iso']} {args['hora']}\n"
        f"convênio={known.get('convênio')}\nLia · ts={ts}",
    )

```

Arquivo 5 — `voice_agent/kommo.py` (PATCH com GET de verificação)

Adicionar após linha 913:

```

def patch_custom_fields_raw(self, lead_id: int, cfs: list[dict]) -> tuple[bool, dict]:
    """PATCH direto custom_fields_values. Bypass de wrapper-mentiroso (Bug C-12).
    Faz GET imediato e CONFERE que field_ids esperados estão presentes.
    """
    payload = {"custom_fields_values": cfs}
    with httpx.Client(timeout=self.timeout) as c:
        r = c.patch(f"{self._base}/leads/{lead_id}", json=payload, headers=self._headers)
        body = {}
        try: body = r.json()
        except Exception: body = {"raw": r.text[:500]}
        ok_2xx = r.status_code // 100 == 2
        if ok_2xx:
            g = c.get(f"{self._base}/leads/{lead_id}", headers=self._headers)
            got_ids = {cf["field_id"] for cf in (g.json().get("custom_fields_values") or [])}
            expected = {c["field_id"] for c in cfs}
            if not expected.issubset(got_ids):
                log.error("[C-12] PATCH 2xx mas campos %s NÃO gravados!", expected - got_ids)
                return False, {"bug": "C-12", "missing": list(expected - got_ids)}
    return ok_2xx, body

```

Fix Bug #240 (User-Agent ausente): alterar `_headers` em `kommo.py:696`:

```

self._headers = {
    "Authorization": f"Bearer {token}",
    "Accept": "application/json",
    "User-Agent": "blink-agent/1.0 (+https://blinkoftalmologia.com.br)", # FIX #240
}

```


Arquivo 6 — `voice_agent/webhook.py` (endpoint leads abandonados + Slack)

```
@app.get("/admin/leads-abandonados")
def leads_abandonados(request: Request) -> JSONResponse:
    if settings.webhook_secret and request.query_params.get("secret") != settings.webhook_secret:
        raise HTTPException(401)
    STATUS_ATIVOS_IA = [
        96441724, 106919911, 101508307, 102560495, 106184631,
        101507507, 101109455, 106653499, 106184983,
    ]
    kc = pipeline.kommo
    abandonados = []
    agora = int(time.time())
    for sid in STATUS_ATIVOS_IA:
        for ld in kc.list_leads_by_status(8601819, [sid], limit=50):
            full = kc.get_lead_full(ld["id"])
            criado = full.get("created_at", agora)
            if agora - criado < 600: continue # < 10min
            if (full.get("notes_count") or 0) > 0: continue
            cfs = full.get("custom_fields_values", [])
            ativ = next((cf for cf in cfs if cf["field_id"] == 1260817), None)
            if ativ and ativ["values"][0].get("value") == "Ativado": continue
            abandonados.append({
                "id": ld["id"], "idade_min": (agora - criado) // 60,
                "status": sid,
            })
    cor = "vermelho" if len(abandonados) >= 3 else ("amarelo" if abandonados else "verde")
    if cor != "verde" and os.getenv("SLACK_WEBHOOK_ALERTAS"):
        httpx.post(os.getenv("SLACK_WEBHOOK_ALERTAS"), json={
            "text": f":rotating_light: [{cor.upper()}] {len(abandonados)} leads abandonados – "
                    + ", ".join(
                        f"<https://univeja.kommo.com/leads/detail/{a['id']}|{a['id']}> "
                        f"({a['idade_min']}min)"
                        for a in abandonados[:5]
                    )
        })
    return JSONResponse({"cor": cor, "total": len(abandonados), "leads": abandonados})
```

Cron Easypanel: `* / 5 * * * * curl -fsS /admin/leads-abandonados?secret=$WEBHOOK_SECRET`.

Arquivo 7 — `tests/test_e2e_apresentar_e_gravar.py` (novo, 8 cenários)

(ver bloco completo no parecer #5 — `tests/test_e2e_apresentar_e_gravar.py` + `tests/test_property_dia_semana.py`)

Arquivo 8 — `voice_agent/smoke_e2e_prod.py` (novo, cron 1h)

```
import os, httpx, json
BASE = os.getenv("SMOKE_BASE_URL", "https://blink-agent.6prkfn.easypanel.host")
SEC = os.getenv("WEBHOOK_SECRET")
SLACK = os.getenv("SLACK_WEBHOOK_SMOKE_URL")

CENARIOS = [
    ("C1_saudacao", {"nome": "Marina"}, "oi",
     lambda t: "marina" in t.lower()),
    ("C2_pediatrico", {"nome": "Bia", "idade_anos": 4}, "quero consulta",
     lambda t: "responsável" in t.lower() or "criança" in t.lower()),
    ("C3_evasiva", {"nome": "Carla"}, "vc atende quando?",
     lambda t: "horário comercial" not in t.lower() and "seg-sex" not in t.lower()),
    ("C4_amil", {"nome": "Diego", "convenio": "Amil"}, "tem amil?",
     lambda t: "particular" in t.lower() or "não atendemos" in t.lower()),
    ("C5_remarcacao", {"nome": "Eva", "status_id": 106184631}, "preciso remarcar",
     lambda t: "manhã" in t.lower() or "tarde" in t.lower() or "/" in t),
]

def run():
    falhas = []
    for nome, known, msg, ok in CENARIOS:
        r = httpx.post(
            f"{BASE}/admin/simulate-inbound?secret={SEC}",
            json={"convo_key": f"smoke-{nome}", "known": known, "inbound": msg},
            timeout=30,
        ).json()
        if not ok(r.get("text", "")):
            falhas.append((nome, r.get("text", "")[:200]))
    if falhas and SLACK:
        httpx.post(SLACK, json={
            "text": f":rotating_light: SMOKE E2E PROD: {len(falhas)} falhas\n"
                  + "\n".join(f"- {n}: {t}" for n, t in falhas)
        })
    return {"total": len(CENARIOS), "falhas": len(falhas), "detalhes": falhas}

if __name__ == "__main__":
    print(json.dumps(run(), indent=2, ensure_ascii=False))
```

Arquivo 9 — `scripts/hooks/pre-commit` (novo)

```
#!/usr/bin/env bash
msg=$(git log -1 --format=%s 2>/dev/null; cat .git/COMMIT_EDITMSG 2>/dev/null)
echo "$msg" | grep -qE '^fix\(bug\):' || exit 0

staged=$(git diff --cached --name-only)
echo "$staged" | grep -qE '^tests/test_.*\.py$' || {
    echo "[BLOCK] commit 'fix(bug):' exige teste em tests/ com input REAL"
    exit 1
}
echo "$staged" | xargs grep -l "lead_id|convo_key.*bug" 2>/dev/null | grep -q . || {
    echo "[BLOCK] teste precisa citar lead_id ou convo_key do caso real"
    exit 1
}
```

Arquivo 10 — `.github/workflows/test.yml` (smoke prod gate)

```
smoke-prod:
  needs: pytest
  runs-on: ubuntu-latest
  steps:
    - name: Smoke /admin/healthz prod
      run: |
        R=$(curl -fsS "https://blink-agent.6prkfn.easypanel.host/admin/healthz?secret=${{ secrets.WEBHOOK_SECRET }}" \
        echo "$R" | jq -e '.integrations.kommo==true and .integrations.medware==true and .integrations.w')
    - name: Smoke /admin/healthz-kommo
      run: |
        curl -fsS "https://blink-agent.6prkfn.easypanel.host/admin/healthz-kommo" \
        | jq -e '.leads_basic.status==200' || (echo "Bug #240"; exit 1)
    - name: Smoke leads-abandonados
      run: |
        curl -fsS "https://blink-agent.6prkfn.easypanel.host/admin/leads-abandonados?secret=${{ secrets.WEBHOOK_SECRET }}" \
        | jq -e '.cor!="vermelho"' || (echo "PROD com leads abandonados"; exit 1)
```

Arquivo 11 (opcional, alto impacto) — `voice_agent/agendamento_agent.py` (specialist agent)

```

"""Agente DEDICADO de agendamento. Pattern router→specialist.
Reduz contexto de ~15k pra ~2k tokens, taxa de tool_use esperada > 95%.
Roteado quando ctx.fsm.estado ∈ {AGENDA, CONFIRMACAO, GRAVACAO}.
"""
from anthropic import Anthropic
from voice_agent.tools_lia import (
    TOOL_OFERECER_SLOT, TOOL_GRAVAR_AGENDAMENTO_MEDWARE,
    handle_oferecer_slot, handle_gravar_agendamento_medware,
)

AGENDA_AGENT_SYSTEM = """Você é a Lia em modo AGENDAMENTO. Regra única e absoluta:
SEMPRE chame a tool apropriada. NUNCA escreva texto livre com horário ou data.

Estado AGENDA → chame `oferecer_slot` com `ctx.agenda` reais.
Estado CONFIRMACAO → chame `gravar_agendamento_medware` com dados confirmados.
Sem agenda real disponível → diga literalmente "Deixa eu reconsultar a agenda
aqui, volto em 1 minuto." (zero invenção).

Águas Claras NUNCA atende noite. Karla NUNCA atende sábado/domingo.
Fabrício atende ter+qui em Asa Norte para catarata.
"""

def reply_agenda(ctx: dict, user_text: str) -> dict:
    cli = Anthropic()
    fsm_estado = (ctx.get("fsm") or {}).get("estado", "AGENDA")
    if fsm_estado == "AGENDA":
        tools = [TOOL_OFERECER_SLOT]
        tc = {"type": "tool", "name": "oferecer_slot"}
    elif fsm_estado in ("CONFIRMACAO", "GRAVACAO"):
        tools = [TOOL_GRAVAR_AGENDAMENTO_MEDWARE]
        tc = {"type": "tool", "name": "gravar_agendamento_medware"}
    else:
        tools, tc = [], {}
    sys_msg = AGENDA_AGENT_SYSTEM + "\n\nCONTEXTO_LEAD:\n" + json.dumps({
        "nome": (ctx.get("known") or {}).get("nome_paciente"),
        "medico": (ctx.get("known") or {}).get("medico"),
        "unidade": (ctx.get("known") or {}).get("unidade"),
        "convenio": (ctx.get("known") or {}).get("convenio"),
        "agenda": ctx.get("agenda", []),
    }, ensure_ascii=False)
    response = cli.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=400, system=sys_msg,
        messages=[{"role": "user", "content": user_text}],
        tools=tools, tool_choice=tc, temperature=0.2,
    )
    # Loop tool use
    while response.stop_reason == "tool_use":
        tool_use = next(b for b in response.content if b.type == "tool_use")
        if tool_use.name == "oferecer_slot":
            result = handle_oferecer_slot(tool_use.input, ctx)
        else:
            result = handle_gravar_agendamento_medware(tool_use.input, ctx)
        # devolve resultado pro modelo wrapper com tom humano
        response = cli.messages.create(
            model="claude-haiku-4-5-20251001", max_tokens=400, system=sys_msg,
            messages=[
                {"role": "user", "content": user_text},
                {"role": "assistant", "content": response.content},
                {"role": "user", "content": [{
                    "type": "tool_result", "tool_use_id": tool_use.id,
                    "content": result.texto_para_paciente or result.erro or "",
                }]},
            ],
            tools=tools, temperature=0.2,
        )
    return {"text": "".join(b.text for b in response.content if b.type == "text")}

```

Em `pipeline.py`, no roteador:

```
if (ctx.get("fsm") or {}).get("estado") in ("AGENDA", "CONFIRMACAO", "GRAVACAO"):
    from voice_agent.agendamento_agent import reply_agenda
    out = reply_agenda(ctx, user_text)
else:
    out = responder.reply(ctx, user_text)
```

Plano de implementação (ordem de impacto)

Hoje (1h)

- 1. **Patch User-Agent em `kommo.py`** (Fix Bug #240) — 5 minutos, desbloqueia motor + endpoints batch.
- 2. **`patch_custom_fields_raw` em `kommo.py`** (Fix Bug C-12) — 15 minutos, desbloqueia observabilidade.
- 3. **Endpoint `/admin/leads-abandonados` + cron 5min** — 20 minutos, evita repetir caso lead 24107106.
- 4. **Push tudo isso** — desbloqueia produção.

Amanhã (4h)

- 1. **Patch tool calling forçado por FSM em `responder.py`** — 1h, elimina família inteira de bugs alucinação.
- 2. **3 filtros novos em `filtros_unificados.py` + plugar em `_scrub_prohibited`** — 1h, backstop reforçado.
- 3. **Pipeline lock Redis** (Bug #183) — 30min, elimina race Sabrina/Kamila/lara.
- 4. **`criar_agendamento_seguro` + `_sync_kommo_pos_gravacao`** — 1h, fluxo Medware blindado.
- 5. **Pytest E2E `test_e2e_apresentar_e_gravar.py` 8 cenários + property-based** — 1h.

Esta semana (1 dia)

- 1. **`agendamento_agent.py` dedicado (Haiku 4.5)** — pattern router→specialist, redução 70% tokens, taxa tool_use > 95%.
- 2. **Smoke E2E prod cron 1h + Slack** — alerta proativo.
- 3. **Pre-commit hook + GitHub Action smoke prod gate.**

Critério de sucesso mensurável (sugestão à ombudsman)

Métrica	Meta 7 dias	Meta 30 dias
Bugs C-NN repetidos (mesmo padrão)	0	0
Lead abandonado > 30min sem alerta	0	0
Taxa tool_use em estado AGENDA	> 80%	> 95%
Agendamentos gravados Medware autônomos	> 60%	> 90%
Mensagens reais entregues por sessão Cowork	> 20	> 100

Conclusão dos 5 engenheiros

Os 4 pareceres convergem em: (a) **tool calling forçado por estado FSM** é o vetor de maior impacto — eliminaria família inteira de bugs (Juliene, Pedro Miguel, Priscila, Carol/Alice, Adelia, Daniela). (b) **Bug #240 + C-12** travam quase tudo em prod hoje; fix em < 1h. (c) **Filtros regex** são backstop bom mas insuficiente sozinhos. (d) **Agente dedicado de**

agendamento com Haiku 4.5 é o caminho arquitetural correto pra longo prazo. (e) **Pytest E2E + smoke prod + pre-commit** = guardrails finais que impedem retorno de bugs já resolvidos.

Tempo total estimado pra implementação completa: **2 dias úteis de engenharia focada**.